GraphRAG 검색 실습: 무엇을 만들고 어떻게 진행하나

인덱싱 실습이 만든 그래프와 검색엔진에서 에이전트(Groq)가 툴을 골라 근거를 모으고, 판단은 Jev가, 막는 일은 코드가 맡음

만드는 것(교재 64쪽 워크플로우)

① 요청 접수: 권한, 같은 세대, 보고서 유무를 확인하고 볼 수 있는 청크 표를 만듦

② 질문 분석: 코드가 회원번호와 카드명을 필터로 바꾸고, Jev 확률로 코드가 잡담, 모호, 첫 툴을 정함

③ 검색 에이전트 ⇄ ④ 툴 ⇄ ⑤ 채점: create_agent + 미들웨어 2개, 최대 6회차

⑥ 리랭킹: 모든 회차 원문을 원 질문으로 다시 채점해 8자리(주제마다 1자리 먼저)

⑦ 수집 종료 판정: 코드 재확인(한 문장 공존 또는 관계 하나) 뒤 Jev로 한 번 더

⑧ 답변 ⇄ ⑨ 검증 → ⑩ 종료: LCEL 체인 + 구조화 출력, 인용과 숫자 검사, 종료 상태 7가지

수업 흐름(4H)

교시	내용	실습 함수
1 이론	교재 64 ~ 79쪽	-
2	① ~ ④ 래퍼	4개
3	④ O, L과 ⑤	4개 + 선택 1
4	⑥ ~ ⑩	5개

실습 방식

- 13개 함수(직접 7, 빈칸 6)를 my_search.py에 채워 바꿔 끼움
- 안 채운 함수는 강사 코드가 대신 돌아 늘 ⑩까지 완주
- 실행마다 에이전트(Groq) 실제 비용이 찍힘(질문 하나 약 $0.001 ~ 0.005)

툴 입력에는 필터 칸이 없음. 세대, 역할, 회원, 카드 필터는 래퍼가 강제로 붙여 에이전트가 넘을 수 없음(교재 69쪽)

상세 보기 →

출처: 『신한카드 하이브리드AI_W05 ~ W07』 64 ~ 79쪽, teacher/plan/graphrag-search-개발계획서.md 4장

무단전재 및 배포 금지

사전 준비 4가지와 수업에서 쓰는 명령 2개

시작 검사가 활성 세대, Neo4j, OpenSearch, 키 3개를 한 번에 확인하고 모자란 것을 알려 줌

사전 준비(수업 전)

1. 인덱싱 실습 완료(활성 세대, 청크 260개). 없으면 --reuse data/teacher
2. Docker로 Neo4j와 OpenSearch 기동(healthy 확인)
3. graphrag 폴더의 공용 가상환경 활성화 (graphrag/README.md 3절)
4. secrets/.env에 키 3줄: JEV, GROQ, COHERE

수업용 명령 2개(활성화 뒤 retriever 폴더에서)

python search_graph.py --qid Q01

약관 사실 질문을 ① ~ ⑩ 끝까지(약 7 ~ 10초). 「답변 완료」와 에이전트 비용 줄을 봄

python search_graph.py --qid Q19

함정 질문(가족돌봄 카드에 없는 해외 수수료 할인)이 「확인할 수 없음」으로 멈춤. 프롬프트에 (.venv)가 없으면 먼저 활성화

출력 줄	봐야 할 것	Q01(강사 PC)
② 코드의 결정	Jev 확률 → 첫 툴 후보	local_search
⑦ 코드 재확인	조건 ①, ②	조건 ① 통과(3문장)
⑩ 종료 상태	이름, 이유, 비용	답변 완료, 약 $0.001

temperature 0이어도 LLM은 실행마다 고르는 툴이 다를 수 있음. 강사와는 신호, 판정, 종료 상태로 비교함

상세 보기 →

출처: hands-on/solutions/graphrag/retriever/README.md 3 ~ 5장, 2026-10-25 강사 PC 실측

무단전재 및 배포 금지

교시 2: ① ~ ④ 래퍼(실습 함수 4개)

결정 규칙은 직접 작성, create_agent와 래퍼처럼 「무엇을 어떤 순서로 부르나」가 개념인 곳은 빈칸 채우기로 둠

단계	함수	방식	채울 것(실습 개념)	교재
①	check_ready	직접	역할 확인, 인덱스와 그래프가 같은 세대, 보고서가 없으면 global_search만 막음	65
②	decide_question_route	직접	Noul 0.7 이상이면 끝냄, Choice는 보기 순서를 바꿔도 같고 확신 0.5 이상일 때만 후보	65, 66
③	build_agent	빈칸 2	create_agent(모델, 툴, 프롬프트, 미들웨어), agent.invoke(입력, context, config)	68
④	run_tool	빈칸 2	질의 속 회원번호 지우기, 필터를 붙여 서브 워크플로우 부르기	69

교시 끝에 확인할 출력

- ② 잡담 질문은 「검색 불필요」로 검색 0회
- ③ 회차 줄과 에이전트가 고른 툴
- ④ Q14: 질의에 회원번호 없음, 다른 회원 원문 0건

보는 포인트

- Jev는 확률만, 결정은 코드 기준값
- 애매한 값(0.3 ~ 0.7)은 끝내지 않고 검색
- 미들웨어: before_model은 상한, wrap_tool_call은 래퍼와 채점

> ***같은 질문을 보기 순서만 바꿔 한 번 더 묻는 까닭은 Jev가 첫 보기로 쏠릴 수 있다고 공식 문서가 밝혀서임***

상세 보기 →

출처: hands-on/solutions/graphrag/retriever/core_search.py, 교재 65 ~ 69쪽

무단전재 및 배포 금지

교시 3: ④ 검색엔진과 Local 검색, ⑤ 채점과 상한(실습 함수 4개 + 선택 1개)

순위 합치기, 엔티티 줄이기, 강제 종료는 순수 파이썬 논리라 직접 작성이고, 신호 규칙만 빈칸 채우기임

단계	함수	방식	채울 것(실습 개념)	교재
④ O	rrf_merge	직접	청크마다 Σ 1 / (60 + 순위), 점수 내림차순 상위 25	70
④ L	narrow_entities	직접	고정 엔티티 → 청크마다 1개 → 유사도 순, 합계 16	71
⑤	diagnose_signals	빈칸 3	NO_SEED, NO_COOCCUR, PASS 규칙(나머지 4개는 완성)	74
⑤	force_stop_reason	직접	회차 6, NO_NEW 2회 연속, 같은 호출 2회 연속, 전량 실패 2회	75
④ E	normalize_entity_key	선택	레이블마다 키를 같은 꼴로(카드 접미어, 조항 번호)	73

교시 끝에 확인할 출력

- Q05: 답이 강사와 같은 조항(제21조)
- Q09: 엔티티 16개 이하
- Q18: NO_COOCCUR가 한 번 이상

보는 포인트

- RRF는 점수 단위가 달라도 순위만 씀
- 세는 일은 Jev가 아니라 코드가 함
- 강제 종료면 모델을 다시 부르지 않음

진단 신호와 기준값은 이 설계의 가정값이고, Jev 판정 기준값은 TypeSafe 공식 쿡북의 출발값임(평가셋으로 조정)

상세 보기 →

출처: hands-on/solutions/graphrag/retriever/core_search.py, 교재 70 ~ 75쪽

무단전재 및 배포 금지

교시 4: ⑥ 리랭킹부터 ⑩ 종료 상태까지(실습 함수 5개)

자리 나누기와 근거 검증은 직접 작성, 재확인 조건과 LCEL 체인, 종료 상태 표는 빈칸 채우기임

단계	함수	방식	채울 것(실습 개념)	교재
⑥	select_final_chunks	직접	주제마다 1자리 먼저, 남은 자리는 점수 순, 8자리	76
⑦	recheck_question	빈칸 2	조건 ① 공존 문장 수, 조건 ② 두 엔티티가 관계 하나로 이어짐	77
⑧	generate_answer	빈칸 2	with_structured_output(GroundedAnswer), LCEL 체인 (프롬프트 → 구조화 모델)	78
⑨	verify_answer	직접	인용 id 존재, 숫자와 조항 번호, 보고서 인용은 경향 문장에만	78
⑩	make_end_state	빈칸 1	END_STATES에서 이름과 뜻 꺼내기	79

교시 끝에 확인할 출력

- Q01: 조건 ① 통과, 검증 통과
- Q18: ⑦ 불통과 → 확인할 수 없음
- Q25: 검증 실패 사유와 재작성이 나올 수 있음

보는 포인트

- 코드 재확인을 Jev가 살리지 않음
- 전제 충돌은 통과 + 표시
- 「확인할 수 없음」도 정상 종료

함정 5문항 × 3회 실측(에이전트 Groq)에서 15번 모두 「확인할 수 없음」이고 15번 모두 ⑦ 코드 재확인과 Jev가 막음

상세 보기 →

출처: hands-on/solutions/graphrag/retriever/core_search.py, 교재 76 ~ 79쪽, 2026-10-25 강사 PC 실측

무단전재 및 배포 금지

본인 코드로 바꿔 끼우기: import 한 줄과 USE_TEACHER_CODE 목록

안 채운 함수는 강사 코드가 대신 돌므로 한 함수씩 채우며 전체를 끝까지 돌려 볼 수 있음

교체 절차

1. search_graph.py의 import 한 줄을 import my_search as core 로 바꿈
2. my_search.py 맨 아래 USE_TEACHER_CODE 목록에서 지금 채울 함수 이름을 지움
3. 그 함수를 채우고 확인 명령(--qid Q01 등)으로 실행함
4. 신호, ⑦ 판정, 종료 상태를 강사 결과와 비교함(툴 경로는 다를 수 있음)
5. 막히면 이름을 목록에 다시 넣고 다음 함수로 넘어감

완료 기준

- USE_TEACHER_CODE가 빈 my_search.py로 Q01이 「답변 완료」, ⑨ 검증 통과
- Q19가 「확인할 수 없음」, 어느 단계에서 막혔는지 설명할 수 있음
- Q14에서 다른 회원 원문 0건
- 제출: my_search.py와 Q01, Q19 실행 화면 캡처

막혔을 때

- NotImplementedError 빈칸 N: 아직 안 채움 또는 raise 줄을 안 지움
- [실습 함수 오류]와 Traceback: 본인 함수의 예외임
- 429(호출 제한): 잠시 뒤 다시 실행
- 강사와 툴 경로가 다름: 정상(신호와 판정으로 비교)

본인 함수에서 예외가 나면 감추지 않고 Traceback을 보여 주고 그 요청만 ⑩ 오류로 끝냄. 고리는 다시 돌지 않음

상세 보기 →

출처: hands-on/solutions/graphrag/retriever/my_search.py, README.md 6장과 9장

무단전재 및 배포 금지